

03 — Python data types through a Case 1 profit equation

The question

As a beginner, I want to understand Python's different kinds of variables, such as `float` and `tuple`, rather than just naming conventions. Can we cover the main data types, inspect their size where applicable, and connect them to a real equation and function from Case 1?

This chapter covers the main built-in value types, NumPy arrays, and a short tour of other objects found in research code. Python also permits user-defined and library-defined types, so there is no finite list of every possible type. Learn the numeric and container sections first; the binary and further-object sections are reference material.

Run [the complete example](#) with Python 3.9+ and NumPy: `python examples/03_python_data_types.py`. It also contains numbered Spyder cells. The small inputs are teaching data, not a reproduction of the paper's optimal decisions. The example was checked with Python 3.9.25 and NumPy 1.26.4.

1. Name, object, type, and value

```
noise_std = 0.12
print(type(noise_std))    # <class 'float'>
print(noise_std)          # 0.12
```

`noise_std` is a name bound to an object; that object has type `float` and value `0.12`. `float` is itself the name of a built-in type, not a declaration attached to `noise_std`. Assigning `noise_std = "unknown"` later binds that name to a string. It does not convert the previous float object into a string.

Use `type(value)` to inspect the exact type and `isinstance(value, float)` to ask whether an object belongs to a type or its subclasses. `isinstance(True, int)` is also true because `bool` subclasses `int`. Avoid using built-in names such as `list`, `str`, or `float` for your own variables: `list = [1, 2]` hides the constructor `list()` in that scope.

A type annotation such as `noise_std: float = 0.12` communicates an intended type. By itself, it neither converts the value nor enforces the type at runtime.

2. Numbers, text, and missing values

These objects do not allow their contents to be changed in place. Reassigning a name is still allowed.

Type	Create a value	Inspect or use it
int	count = 3	count + 1; an integer count
float	noise_std = 0.12	noise_std ** 2; a real-valued approximation
complex	z = 2 + 3j	z.real, z.imag, abs(z)
bool	converged = True	if converged:: a logical condition
str	label = "Case 1"	label[0] gives "C"; len(label) gives 6
NoneType	result = None	result is None; no result yet

Python float normally uses binary double precision. Decimal fractions may not be exact: `0.1 + 0.2 == 0.3` is false. Use `math.isclose` or `np.isclose` with tolerances suited to the problem. None is not zero, an empty string, or NumPy's numerical nan; `type(None)` gives `NoneType`. Numerical nan is a floating-point value, and `np.isnan` checks for it.

```
float("0.12")    # 0.12: parse text
int(3.9)          # 3: truncate toward zero
str(3)            # "3": produce text
bool(0)           # False
bool("False")     # True: a nonempty string
```

`len(3)` and `0.12.shape` are invalid. Scalar values are not one-element sequences. `len("你好")` is 2 Unicode code points; encoded byte length is a different quantity.

3. Sequences: list, tuple, and range

Type	Construction and access	Can its slots change?
list	q = [1.0, 2.0, 3.0]; q[0]	Yes: q[0] = 4.0, q.append(5.0)
tuple	bounds = (0.0, 6.0); bounds[1]	No reassignment of tuple slots
range	indices = range(3); indices[0]	No; describes 0, 1, 2 without storing a list

Indexing starts at zero. `q[-1]` selects the last item; `q[0:2]` selects the first two. `len(q)` counts top-level items. Lists and tuples can contain mixed types, for example `("seed", 5, True)`, though numerical calculations usually benefit from homogeneous arrays.

```
single = (3,)          # a one-item tuple
not_a_tuple = (3)      # an int
lower, upper = (0.0, 6.0) # unpack two values
values = list(range(3)) # [0, 1, 2]
```

The comma makes a one-item tuple. In Q1, `return x, y` returns a tuple of two arrays, which the caller unpacks. An array's shape, such as `(4, 3)`, is also a tuple.

4. Mapping and sets

Type	Construction	Access and purpose
dict	<code>cfg = {"seed": 5, "std": 0.12}</code>	<code>cfg["seed"]</code> ; named settings
set	<code>ids = {1, 2, 2}</code>	<code>2 in ids</code> ; unique members
frozenset	<code>fixed = frozenset({1, 2})</code>	<code>2 in fixed</code> ; immutable set

A dictionary maps unique, hashable keys to values; `len(cfg)` counts entries. `cfg["seed"] = 6` updates a value. `cfg.get("missing")` returns `None` by default, while `cfg["missing"]` raises `KeyError`. Dictionaries preserve insertion order. Sets have no positional indexing or promised iteration order; `ids[0]` is invalid. Use `ids.add(3)` to modify a set.

`{}` is an empty dictionary; `set()` is an empty set. Hashability means an object can safely serve as a dictionary key or set member under Python's hashing rules. Lists cannot; tuples can only when all their elements are hashable. Do not assume that every immutable container is hashable regardless of its contents.

5. Binary values: useful when reading files or buffers

Type	Example	Access and mutability
bytes	<code>raw = b"ABC"</code>	<code>raw[0] == 65</code> ; immutable bytes
bytearray	<code>buf = bytearray(b"ABC")</code>	<code>buf[0] = 90</code> ; mutable bytes
memoryview	<code>view = memoryview(buf)</code>	<code>view[0] = 90</code> modifies the backing buffer

For these one-dimensional byte examples, `len()` is 3. `raw.decode("utf-8")` produces text; `"ABC".encode("utf-8")` produces bytes. A memory view exposes an existing buffer without copying its data. Writability depends on the backing object: a view over bytes is read-only. More general memory views can have their own shape, format, and nbytes.

6. Mutation, aliases, and copies

```
q = [1.0, 2.0]
alias = q
copied = q.copy()
alias[0] = 9.0
# q is now [9.0, 2.0]; copied is still [1.0, 2.0]
```

`alias = q` adds another name for the same list. It does not copy the list. A shallow copy creates a new outer container but shares references to nested objects. A tuple can hold a mutable object: `t = ([1, 2],)` permits `t[0].append(3)`, even though replacing the tuple slot with `t[0] = []` is forbidden.

For numeric NumPy arrays, basic slices normally share data with the original array, while `a.copy()` copies the data. Inspect `np.shares_memory(a, b)` when the relationship matters. An object-dtype array can still contain references to shared Python objects after a shallow array copy.

7. NumPy: type, dtype, shape, size, and len

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]], dtype=np.float64)
```

Inspection	Result	Meaning in this example
<code>type(a)</code>	<code>numpy.ndarray</code>	Type of the whole array object
<code>a.dtype</code>	<code>float64</code>	Element storage type
<code>a.shape</code>	<code>(2, 3)</code>	Two rows, three columns
<code>a.ndim</code>	<code>2</code>	Number of axes
<code>a.size</code>	<code>6</code>	Total number of elements
<code>len(a)</code>	<code>2</code>	Length of the first axis
<code>a.itemsize</code>	<code>8</code>	Bytes per element
<code>a.nbytes</code>	<code>48</code>	Element buffer bytes, excluding object overhead

`type(a[0, 0])` is `np.float64`; `type(a[0, 0].item())` is Python `float`. A NumPy scalar and a zero-dimensional array differ: `np.array(0.12)` is an ndarray with `shape == ()` and `size == 1`, but `len()` is invalid. `float64`, `int64`, and `bool` describe common element dtypes; they do not determine an array's shape.

Lists do not have array shape or dtype. `[1, 2] * 2` repeats a list; `np.array([1, 2]) * 2` multiplies each element. Prefer explicit dtypes for reproducible storage choices; default integer widths can depend on the NumPy version and platform.

8. The real Case 1 function

Source: `Code_Python_rogp39/Case1-Production-Planning/` → `comparative-study-production-planning_v2_revised_eng.ipynb`, section **S6.1**, in the separate GaussianProcess-Integrated-Optimization repository. The following two functions retain the notebook's executable statements. Extra type demonstrations elsewhere in this chapter are teaching additions, not claims about types used by this function.

```
def true_price_fn(x):
    x = np.asarray(x, dtype=float)
    return np.exp(-x / 2.0)

def realized_profit_samples(x, noise_matrix, cost_vec):
    x = np.asarray(x, dtype=float).reshape(1, -1)
    cost_vec = np.asarray(cost_vec, dtype=float).reshape(1, -1)
    realized_price = true_price_fn(x) + noise_matrix
    return np.sum(x * (realized_price - cost_vec), axis=1)
```

The equation below uses plain text so it renders identically in Markdown and offline PDF:

```
price[s, j] = exp(-x[j] / 2) + noise[s, j]
profit[s] = SUM over j of x[j] * (price[s, j] - cost[j])
s = scenario index; j = production component index
```

Python name	Role in the equation	Shape after conversion
x	Production quantities x[j]	(1, J)
cost_vec	Unit costs cost[j]	(1, J)
noise_matrix	Price errors noise[s, j]	(S, J) expected
realized_price	Scenario prices	(S, J)
Returned array	One profit per scenario	(S,)

x, noise_matrix, and cost_vec are parameters; realized_price is a local name. On the first line, `np.asarray(..., dtype=float)` converts a list or tuple to a floating-point array, or may reuse compatible array storage. `reshape(1, -1)` creates one row and infers the column count; -1 is not negative indexing here. Local reassignment of x does not rebind the caller's x_input name.

The helper computes the noiseless price element by element. Adding the (S, J) noise matrix broadcasts the (1, J) prices across scenarios. Subtracting costs and multiplying quantities is elementwise. Finally, `axis=1` sums across columns, leaving one value for each row. Omitting `axis` would combine all scenarios into one scalar. `float` is a type passed as an argument; `dtype` and `axis` are keyword parameter names; `reshape` is a method. These are different roles despite appearing on the same line.

The original function assumes compatible numeric inputs; it does not validate dimensions or enforce economic constraints. In particular, a (S, 1) noise array can broadcast the same shock across every component. Verify the intended (S, J) shape before calling it; successful broadcasting alone does not prove the model is correct.

9. A hand-checkable run and Spyder inspection

The script uses three quantities [1.0, 2.0, 3.0], costs (0.1, 0.2, 0.3), and four explicit noise rows:

```
[ 0.00,  0.00,  0.00]
[ 0.10,  0.00, -0.10]
[-0.10,  0.10,  0.00]
[ 0.02, -0.03,  0.04]
```

The zero-noise profit is approximately 0.61168002. Relative to it, the other scenarios change profit by -0.20, +0.10, and +0.08: multiply each noise row by the three quantities and add. Expected output:

```
profits = [0.61168002 0.41168002 0.71168002 0.69168002]
profit shape = (4,)
Verification passed.
```

1. Run cells 1–3 to inspect scalar and container names in Spyder's Variable Explorer. The `samples` dictionary collects all 15 core types in one place. Some object types may be filtered or lack a graphical editor; `type(value)` in the console remains useful.
2. Place a breakpoint on the return line marked P1, then debug the script. At that point, the preceding assignments have executed and the return has not.
3. Inspect `type(x)`, `x.dtype`, `x.shape`, `noise_matrix.shape`, `realized_price.shape`, and `type(x.shape)`. Expect (1, 3), (4, 3), (4, 3), and a tuple for the shape object.
4. After returning, inspect `profits` and compare with `reference`, calculated using nested loops rather than broadcasting. The script checks their agreement and the scenario profit changes.

10. Other objects you will meet

This is a recognition guide, not a requirement to learn every advanced class now. The final script cell creates inspectable examples without writing files or raising exceptions.

Object family	Example	What to recognize
Exact decimal / rational	<code>Decimal("0.1"), Fraction(1, 3)</code>	Standard-library numeric classes; not built-in float
File-system path	<code>Path("results")</code>	Library object representing a path
Function / module / class	<code>true_price_fn, np, float</code>	Callable function, imported module, type object
Iterator / generator	<code>iter([1, 2]), (i*i for i in range(3))</code>	<code>next()</code> consumes values; usually no <code>len()</code>
Slice	<code>slice(0, 2)</code>	Reusable indexing selection
Exception instance	<code>ValueError("bad input")</code>	An object that can be raised with <code>raise</code>
Special singleton values	<code>Ellipsis, NotImplemented</code>	Extended slicing; unsupported operator protocol

`NotImplemented` is distinct from the exception `NotImplementedError`; it is not a generic missing-value marker. Files, custom class instances, pandas tables, and optimization model objects also have their own types. Inspect their type and documented interface instead of assuming they behave like an array.

11. Practice and source notes

Predict before running: What is the type of `(3)` versus `(3,)`? Why does `len(a)` differ from `a.size`? Why does assigning `alias = a` affect the original list later? Why does a profit array have shape `(4,)` instead of `(4, 3)`? The answers are in sections 3, 7, 6, and 8 respectively.

The [Python built-in types reference](#)^[1] is the lookup source for numeric, sequence, mapping, set, and binary operations. The [NumPy ndarray reference](#)^[2] documents array attributes and methods. The worked inputs, manual profit comparison, and teaching organization here are specific to this handbook.

References

1. <https://docs.python.org/3/library/stdtypes.html>
2. <https://numpy.org/doc/stable/reference/arrays.ndarray.html>